

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Текст програми

ІА/ЛЦ.467200.007 Д4

ПЕРЕЛІК ФАЙЛІВ ПРОГРАМНОГО КОМПЛЕКСУ

У цьому додатку наведено повні вихідні тексти ключових модулів програмного комплексу системи трикутного просторового аналізу польоту волейбольного м'яча. Модулі впорядковано відповідно до архітектурного графа залежностей, описаного в розділі 3 пояснювальної записки: спочатку наводиться модуль геометричного зворотного проектування, далі — ядро байєсової фільтрації, потім модуль трекінгу зі скінченням автоматом життєвого циклу треку та, нарешті, сценарій навчання згорткової нейромережі-детектора. Сумарний обсяг наведеного коду становить понад 950 рядків мовою програмування Python.

ball_3d_analysis/get_court_position_methods.py

```
import cv2
import numpy as np
import torch as t
import torchvision
import torchaudio
import albumentations as A
from ultralytics import YOLO
```

Reference calibration correspondences: the four court corners.

Court is 9 m wide x 18 m long; world coordinates are in metres.

```
pts_real_3d = np.array([
    [0.0, 0.0, 0.0],
```

					ІАЛЦ.467200.007 Д4			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Безщасний Р.Р.				Система просторового аналізу гри в волейбол з використанням машинного навчання		Лит.	Аркуш
Перевірів	Морозов-Леонів							78
							НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-21	
Н. контр.								
Затвердив	Волокита А. М.							

```
[9.0, 0.0, 0.0],
[9.0, 0.0, 18.0],
[0.0, 0.0, 18.0]],dtype=np.float32)
```

Pixel locations of the same four corners in the broadcast frame.

```
pts_video_2d = np.array([
    [ 69, 1033], # P0 front-left
    [1840, 1031], # P1 front-right
    [1507, 609], # P2 back-right
    [ 405, 609]], # P3 back-left
    dtype=np.float32)
```

Initial pinhole intrinsics (principal point at the image centre). The focal
length is only a starting guess here — it is refined by calibration.

```
K = np.array([
    [1300.0, 0.0, 960.0],
    [0.0, 1300.0, 540.0],
    [0.0, 0.0, 1.0]],dtype=np.float32)
```

```
def calibrate_camera(pts_3d, pts_2d, camera_matrix, dist, up_axis=1):
```

```
    """
```

Calibrate the camera with PnP. Returns (R, tvec, camera_position):

R — world->camera rotation matrix (3x3),

tvec — translation into the camera frame (3,1),

camera_position — camera location in world coordinates (3,1).

up_axis flips R so the camera ends up above the floor: the four coplanar

court corners leave the vertical direction sign-ambiguous otherwise.

''''''

```
success, rvec, tvec = cv2.solvePnP(pts_3d, pts_2d, camera_matrix, dist,
flags=cv2.SOLVEPNP_ITERATIVE)
```

```
if not success:
```

```
    raise ValueError("solvePnP failed to converge; "
                    "check the order of the point correspondences")
```

```
R, _ = cv2.Rodrigues(rvec)
```

```
# Camera position in world coordinates (C = -R^T * tvec).
```

```
R_inv = np.linalg.inv(R)
```

```
camera_position = -np.dot(R_inv, tvec)
```

```
if (up_axis is not None
```

```
    and 0 <= up_axis < 3
```

```
    and camera_position[up_axis, 0] < 0.0):
```

```
    flip = np.eye(3)
```

```
    flip[up_axis, up_axis] = -1.0
```

```
    R = np.dot(R, flip)
```

```
    camera_position = np.dot(flip, camera_position)
```

```
return R, tvec, camera_position
```

```
def get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos,
```

```
    ball_diameter=0.21, z_min=2.0, z_max=25.0,
```

```
    return_none_on_clip=False):
```

					ІА/Ц.467200.007 Д4	Аркуш
						3
Зм.	Лист	№ докум.	Підп.	Дата		

''''''

Back-project a ball detection into a 3D world position.

Args:

u, v: ball bbox centre, in pixels.

bbox_w: ball bbox width, in pixels. Width is used (not height)
because it suffers less from motion blur.

K: camera intrinsics matrix.

R_matrix: rotation matrix from solvePnP.

camera_pos: 3D camera position (C) from solvePnP.

ball_diameter: real volleyball diameter, in metres (21 cm nominal).

z_min, z_max: sanity bounds on Z_c (distance from camera along the ray).

Returns:

np.ndarray (3,) — world position [X, Y, Z]; or None when

return_none_on_clip=True and Z_c fell outside [z_min, z_max].

''''''

f_x = K[0, 0]

c_x = K[0, 2]

f_y = K[1, 1]

c_y = K[1, 2]

Z_c = (f_x * ball_diameter) / bbox_w

if Z_c < z_min or Z_c > z_max:

if return_none_on_clip:

return None

Z_c = max(z_min, min(z_max, Z_c))

```

x_norm = (u - c_x) / f_x
y_norm = (v - c_y) / f_y
d_cam = np.array([[x_norm], [y_norm], [1.0]])

d_world = np.dot(R_matrix.T, d_cam)
P_world = camera_pos.reshape(3, 1) + Z_c * d_world

return P_world.ravel()

```

```

def get_3d_position_with_cov(u, v, bbox_w, K, R_matrix, camera_pos,
    ball_diameter=0.21, sigma_w=1.0, sigma_uv=1.0, r_floor=0.02,
    z_min=2.0, z_max=25.0, return_none_on_clip=False):
    """

```

Like get_3d_position, but also returns a 3x3 measurement covariance R in world coordinates — anisotropic and physically consistent with the monocular depth geometry.

Depth along the ray is $Z_c = f \cdot D / w_{\text{box}}$, so its error is driven by the bbox-width error σ_w through linearisation:

$$\sigma_{Z_c} = |dZ_c/dw| * \sigma_w = (f \cdot D / w_{\text{box}}^2) * \sigma_w.$$

σ_{Z_c} grows with f and with $1/w^2$ — i.e. depth deserves less trust for a larger (correct) focal length and for smaller (distant) balls. This is the honest replacement for a fixed $R_z = 0.5$.

The uncertainty lies along the ray (direction d_{world}), not along the world

Z axis, so the world covariance is mostly a rank-1 ellipsoid stretched along the ray:

$$\begin{aligned} \text{Cov} \sim & \sigma_z^2 * d_{\text{world}} * d_{\text{world}}^T \quad (\text{dominant, depth}) \\ & + Z_c^2 * \sigma_u^2 * J_u * J_u^T \quad (\text{lateral, from pixel } u) \\ & + Z_c^2 * \sigma_v^2 * J_v * J_v^T \quad (\text{lateral, from pixel } v) \end{aligned}$$

with $J_u = R^T * [1/f_x, 0, 0]^T$, $J_v = R^T * [0, 1/f_y, 0]^T$ — the shift of the point per 1 px move of the bbox centre.

Args:

`sigma_w`: bbox-width sigma, in pixels (detector noise). Default 1.0.

`sigma_uv`: bbox-centre sigma in u,v, in pixels. Default 1.0.

`r_floor`: isotropic variance floor (m^2) added to every axis.

Default 0.02, matching the lateral term of the old fixed

$R = \text{diag}([0.02, 0.02, 0.5])$. Keeps lateral trust from collapsing

below the proven value (monocular localises sideways very well, so

the rank-1 term leaves the lateral eigenvalues tiny) and guarantees

positive-definiteness. Only the along-ray depth stays adaptive

(σ_z^2 on top of the floor).

Returns:

(`P_world (3,)`, `R_cov (3,3)`); or (`None`, `None`) on clip when

`return_none_on_clip=True`.

~~~~~

`f_x = K[0, 0]`

`c_x = K[0, 2]`

`f_y = K[1, 1]`

$c_y = K[1, 2]$

$Z_c = (f_x * \text{ball\_diameter}) / \text{bbox\_w}$

if  $Z_c < z_{\min}$  or  $Z_c > z_{\max}$ :

if return\_none\_on\_clip:

return None, None

$Z_c = \max(z_{\min}, \min(z_{\max}, Z_c))$

$x_{\text{norm}} = (u - c_x) / f_x$

$y_{\text{norm}} = (v - c_y) / f_y$

$d_{\text{cam}} = \text{np.array}([x_{\text{norm}}, y_{\text{norm}}, 1.0])$

$d_{\text{world}} = \text{np.dot}(R_{\text{matrix.T}}, d_{\text{cam}})$  # (3,1) ray direction

$P_{\text{world}} = \text{camera\_pos.reshape}(3, 1) + Z_c * d_{\text{world}}$

# Depth sigma along the ray, from the bbox-width error.

$\text{sigma\_Zc} = (f_x * \text{ball\_diameter}) / (\text{bbox\_w} ** 2) * \text{sigma\_w}$

# Lateral Jacobians (point shift per 1 px move of the bbox centre).

$j_u = \text{np.dot}(R_{\text{matrix.T}}, \text{np.array}([1.0 / f_x, 0.0, 0.0]))$

$j_v = \text{np.dot}(R_{\text{matrix.T}}, \text{np.array}([0.0, 1.0 / f_y, 0.0]))$

$R_{\text{cov}} = (\text{sigma\_Zc} ** 2) * \text{np.dot}(d_{\text{world}}, d_{\text{world.T}})$

$R_{\text{cov}} += (Z_c ** 2) * (\text{sigma\_uv} ** 2) * \text{np.dot}(j_u, j_u.T)$

$R_{\text{cov}} += (Z_c ** 2) * (\text{sigma\_uv} ** 2) * \text{np.dot}(j_v, j_v.T)$

# Isotropic floor: keeps lateral trust at the proven level of the old R



```
# (0.02) and guarantees positive-definiteness. Only the along-ray depth
# stays adaptive (sigma_Zc^2 on top of the floor).
R_cov += np.eye(3) * r_floor

return P_world.ravel(), R_cov.astype(np.float64)
```

### **ball\_3d\_analysis/IMM\_UKF.py**

```
import numpy as np
from copy import deepcopy
from scipy.linalg import cholesky, inv, expm, block_diag
from get_court_position_methods import calibrate_camera, get_3d_position
from scipy.stats import multivariate_normal
import sys
```

```
class UnscentedKalmanFilter(object):
```

```
    def __init__(self, name, dim_x, dim_z, dt, fx, hx, points, sqrt_fn=None,
                  x_mean_fn=None, z_mean_fn=None, residual_x=None, residual_z=None):
        self.name = name
        self.x = np.zeros(dim_x)
        self.P = np.eye(dim_x)
        self.x_prior = np.copy(self.x)
        self.P_prior = np.copy(self.P)
        self.Q = np.eye(dim_x)
        self.R = np.eye(dim_z)
        self._dim_x = dim_x
```

```

self._dim_z = dim_z
self.points_fn = points
self._dt = dt
self._num_sigmas = points.num_sigmas()
self.hx = hx
self.fx = fx
self.x_mean_fn = x_mean_fn
self.z_mean_fn = z_mean_fn
self.Wm, self.Wc = points.Wm, points.Wc
self.likelihood = None
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

if sqrt_fn is None:
    self.msqrt = cholesky
else:
    self.msqrt = sqrt_fn

if residual_x is None:
    self.residual_x = np.subtract
else:
    self.residual_x = residual_x

if residual_z is None:
    self.residual_z = np.subtract
else:
    self.residual_z = residual_z

```

```

# sigma points transformed through f(x) and h(x)
self.sigmas_f = np.zeros((self._num_sigmas, self._dim_x))
self.sigmas_h = np.zeros((self._num_sigmas, self._dim_z))

self.K = np.zeros((dim_x, dim_z))
self.y = np.zeros(dim_z)
self.z = np.array([[None]*dim_z])
self.S = np.zeros((dim_z, dim_z))
self.SI = np.zeros((dim_z, dim_z))
self.inv = np.linalg.inv

self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

self.x_post = self.x.copy()
self.P_post = self.P.copy()

```

```

def predict(self, dt=None, UT=None, fx=None, **fxargs):

```

```

    """

```

```

    :param dt: time step for next iteration of computation

```

```

    :param UT: unscented transform function

```

```

    :param fx: state transition function

```

```

    :param fxargs: arguments passed to f(x)

```

```

    """

```

```

    if dt is None:

```

```

        dt = self._dt

```

```

    if UT is None:

```

```

UT = unscented_transform

# calculating sigma points for given mean and covariance
self.compute_process_sigmas(dt, fx, **fxargs)

#pass sigmas through the unscented transform to compute the prior
self.x, self.P = UT(self.sigmas_f, self.Wm, self.Wc,
                    self.Q, self.x_mean_fn, self.residual_x)

self.P = (self.P + self.P.T) / 2.0
self.x_prior = np.copy(self.x)
self.P_prior = np.copy(self.P)

def update(self, z, R=None, UT=None, hx=None, **hxargs):
    """
    :param z: measurement vector
    :param R: measurement noise matrix
    :param UT: function for unscented transform
    :param hx: measurement function
    :param hxargs: arguments for hx
    """

    if z is None:
        self.z = np.array([[None]*self._dim_z]).T
        self.x_post = self.x.copy()
        self.P_post = self.P.copy()
        self.likelihood = 1.0
        return

    if hx is None:
        hx = self.hx

```

if UT is None:

    UT = unscented\_transform

if R is None:

    R = self.R

elif np.isscalar(R):

    R = np.eye(self.\_dim\_z) \* R

# passing prior sigmas through h(x) to get measurement sigmas

# the shape of simgas\_h will vary if the shape of z varies

sigmas\_h = []

for s in self.sigmas\_f:

    sigmas\_h.append(hx(s, \*\*hxargs))

self.sigmas\_h = np.atleast\_2d(sigmas\_h)

# mean and covariance of prediction passed through unscented transform

z\_mean, self.S = UT(self.sigmas\_h, self.Wm, self.Wc,

    R, self.z\_mean\_fn, self.residual\_z)

# Regularise S before inversion / likelihood: guarantees positive

# definiteness even when the covariance is momentarily degenerate.

S\_stable = self.S + np.eye(self.S.shape[0]) \* 1e-6

try:

    self.SI = np.linalg.inv(S\_stable)

except np.linalg.LinAlgError:

    self.SI = np.linalg.pinv(S\_stable)

```

# compute cross variance of the state and measurement
Pxz = self.cross_variance(self.x, z_mean, self.sigmas_f, self.sigmas_h)

self.K = np.dot(Pxz, self.SI) # Kalman gain
self.y = self.residual_z(z, z_mean)

try:
    self.likelihood = multivariate_normal.pdf(
        self.y,
        mean=np.zeros_like(self.y),
        cov=S_stable,
        allow_singular=True,
    )
except (np.linalg.LinAlgError, ValueError):
    self.likelihood = 1e-300

if not np.isfinite(self.likelihood) or self.likelihood <= 0.0:
    self.likelihood = 1e-300

# updating Gaussian state estimate(x, P)
self.x = self.x + np.dot(self.K, self.y)
self.P = self.P - np.dot(self.K, np.dot(self.S, self.K.T))

# save measurement and posterior state
self.z = deepcopy(z)
self.x_post = self.x.copy()
self.P_post = self.P.copy()

```

```

def cross_variance(self, x, z, sigmas_f, sigmas_h):
    # Computing cross variance of the state 'x' and measurement 'z'
    Pxz = np.zeros((sigmas_f.shape[1], sigmas_h.shape[1]))
    N = sigmas_f.shape[0]
    for i in range(N):
        dx = self.residual_x(sigmas_f[i], x)
        dz = self.residual_z(sigmas_h[i], z)
        Pxz += self.Wc[i] * np.outer(dx, dz)
    return Pxz

def compute_process_sigmas(self, dt, fx=None, **fx_args):
    # computes the values of sigmas_f
    if fx is None:
        fx = self.fx

    # calculate sigma points for given mean and covariance
    sigmas = self.points_fn.sigma_points(self.x, self.P)

    for i, s in enumerate(sigmas):
        self.sigmas_f[i] = fx(s, dt, **fx_args)

@property
def log_likelihood(self):
    """
    log-likelihood of the last measurement.
    """
    if self._log_likelihood is None:
        self._log_likelihood = logpdf(x=self.y, cov=self.S)

```

```
return self._log_likelihood
```

```
@property
```

```
def mahalanobis(self):
```

```
    """
```

```
    Mahalanobis distance of measurement. E.g. 3 means measurement
    was 3 standard deviations away from the predicted value.
```

```
    """
```

```
    if self._mahalanobis is None:
```

```
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
```

```
    return self._mahalanobis
```

```
class IMMEstimator(object):
```

```
    """ Implements an Interacting Multiple-Model (IMM) estimator.
```

```
    :param filters: N-list consisting of filters in sequenced order
```

```
    :param mu: (N) array-like of float with mode probabilities for each filter
```

```
    :param M: (N, N) Markov chain transition matrix
```

```
    """
```

```
    def __init__(self, filters, mu, M):
```

```
        if len(filters) < 2:
```

```
            raise ValueError('filters must contain at least two filters')
```

```
        self.filters = filters
```

```
        self.mu = np.asarray(mu) / np.sum(mu)
```

```
        self.M = M
```



```

x_shape = filters[0].x.shape
for f in filters:
    if x_shape != f.x.shape:
        raise ValueError(
            'All filters must have the same state dimension')

self.x = np.zeros(filters[0].x.shape)
self.P = np.zeros(filters[0].P.shape)
self.N = len(filters)
self.likelihood = np.zeros(self.N)
self.omega = np.zeros((self.N, self.N))
self._compute_mixing_probabilities()

# initialize imm state estimate based on current filters
self._compute_state_estimate()
self.x_prior = self.x.copy()
self.P_prior = self.P.copy()
self.x_post = self.x.copy()
self.P_post = self.P.copy()
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

def filter_setx(self):
    for f in self.filters:
        f.x = self.x

def update(self, z, R=None):
    >>>>>

```

Add a new measurement (z) to the Kalman filter. If z is None, nothing is changed.

:param z: measurement

:param R: optional per-detection measurement covariance (3x3). If None, each filter uses its own fixed self.R. Passing a per-detection R (adaptive sigma\_Z) makes depth trust depend on the bbox width.

''''''

```
# run update on each filter, and save the likelihood
```

```
for i, f in enumerate(self.filters):
```

```
    f.update(z, R=R)
```

```
    self.likelihood[i] = f.likelihood
```

```
# update mode probabilities from total probability * likelihood
```

```
self.mu = self.cbar * self.likelihood
```

```
sum_mu = np.sum(self.mu)
```

```
if sum_mu == 0.0:
```

```
    self.mu = self.cbar.copy()
```

```
else:
```

```
    self.mu /= sum_mu
```

```
self._compute_mixing_probabilities()
```

```
# compute mixed IMM state and covariance and save posterior estimate
```

```
self._compute_state_estimate()
```

```
self.x_post = self.x.copy()
```

```
self.P_post = self.P.copy()
```

```

def predict(self, u=None):
    """
    Predict next state (prior) using the IMM state propagation
    equations.
    :param u: control vector, if None then none
    """

    # compute mixed initial conditions
    xs, Ps = [], []
    for i, (f, w) in enumerate(zip(self.filters, self.omega.T)):
        x = np.zeros(self.x.shape)
        for kf, wj in zip(self.filters, w):
            x += kf.x * wj
        xs.append(x)

        P = np.zeros(self.P.shape)
        for kf, wj in zip(self.filters, w):
            y = kf.x - x
            P += wj * (np.outer(y, y) + kf.P)
        Ps.append(P)

    # compute each filter's prior using the mixed initial conditions
    for i, f in enumerate(self.filters):
        # propagate using the mixed state estimate and covariance
        f.x = xs[i].copy()
        f.P = Ps[i].copy()
        f.predict(u)

```

```

# compute mixed IMM state and covariance and save posterior estimate
self._compute_state_estimate()
self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

def _compute_state_estimate(self):
    """
    Computes the IMM's mixed state estimate from each filter using
    the the mode probability self.mu to weight the estimates.
    """
    self.x.fill(0)
    for f, mu in zip(self.filters, self.mu):
        self.x += f.x * mu

    self.P.fill(0)
    for f, mu in zip(self.filters, self.mu):
        y = f.x - self.x
        self.P += mu * (np.outer(y, y) + f.P)

def _compute_mixing_probabilities(self):
    # Compute the mixing probability for each filter

    self.cbar = np.dot(self.mu, self.M)
    for i in range(self.N):
        for j in range(self.N):
            self.omega[i, j] = (self.M[i, j]*self.mu[i]) / self.cbar[j]

@property

```

```
def log_likelihood(self):
```

```
    """
```

```
    log-likelihood of the last measurement.
```

```
    """
```

```
    if self._log_likelihood is None:
```

```
        self._log_likelihood = logpdf(x=self.y, cov=self.S)
```

```
    return self._log_likelihood
```

```
@property
```

```
def mahalanobis(self):
```

```
    """
```

```
    Mahalanobis distance of measurement. E.g. 3 means measurement  
    was 3 standard deviations away from the predicted value.
```

```
    """
```

```
    if self._mahalanobis is None:
```

```
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
```

```
    return self._mahalanobis
```

```
class MerweScaledSigmaPoints(object):
```

```
    """
```

```
    Generates sigma points and weights according to Van der Merwe's  
    2004 dissertation[1] for the UnscentedKalmanFilter class. It  
    parametrizes the sigma points using alpha, beta, kappa terms, and  
    is the version seen in most publications.
```

:param n : int

Dimensionality of the state.  $2n+1$  weights will be generated.

:param alpha : float

Determines the spread of the sigma points around the mean.

Usually a small positive value ( $1e-3$ ).

:param beta : float

Incorporates prior knowledge of the distribution of the mean. For Gaussian  $\alpha + \beta = 1$  is optimal.

:param kappa : float, default=0.0

Secondary scaling parameter usually set to 0 or to  $3-n$ .

:param sqrt\_method : function(ndarray), default=scipy.linalg.cholesky

Defines how we compute the square root of a matrix, which has no unique answer. Cholesky is the default choice due to its speed (another good choice - `scipy.linalg.sqrtm`).

:param subtract : callable (x, y), optional

Function that computes the difference between x and y.

You will have to supply this if your state variable cannot support subtraction, such as angles ( $359-1$  degrees is 2, not 358). x and y are state vectors, not scalars.

Attributes:

$W_m$  : np.array

weight for each sigma point for the mean

Wc : np.array

weight for each sigma point for the covariance

"""

```
def __init__(self, n, alpha, beta, kappa, sqrt_method=None, subtract=None):
```

```
#pylint: disable=too-many-arguments
```

```
self.n = n
```

```
self.alpha = alpha
```

```
self.beta = beta
```

```
self.kappa = kappa
```

```
if sqrt_method is None:
```

```
    self.sqrt = cholesky
```

```
else:
```

```
    self.sqrt = sqrt_method
```

```
if subtract is None:
```

```
    self.subtract = np.subtract
```

```
else:
```

```
    self.subtract = subtract
```

```
self._compute_weights()
```

```
def num_sigmas(self):
```

```
    """ Number of sigma points for each variable in the state x """
```

```
    return 2*self.n + 1
```

```
def sigma_points(self, x, P):
```

```
    """ Computes the sigma points for an unscented Kalman filter
    given the mean (x) and covariance(P) of the filter.
    Returns tuple of the sigma points and weights.
```

```
    Works with both scalar and array inputs:
```

```
    sigma_points (5, 9, 2) # mean 5, covariance 9
```

```
    sigma_points ([5, 2], 9*eye(2), 2) # means 5 and 2, covariance 9I
```

```
    :param x : An array-like object of the means of length n
```

```
    :param P : scalar, or np.array
```

```
        Covariance of the filter. If scalar, is treated as eye(n)*P.
```

```
    :return np.array, of size (n, 2n+1)
```

```
        Two dimensional array of sigma points. Each column contains all of
        the sigmas for one dimension in the problem space.
```

```
        Ordered by Xi_0, Xi_{1..n}, Xi_{n+1..2n}
```

```
    """
```

```
    if self.n != np.size(x):
```

```
        raise ValueError("expected size(x) {}, but size is {}".format(
            self.n, np.size(x)))
```

```
    n = self.n
```



```
if np.isscalar(x):
```

```
    x = np.asarray([x])
```

```
if np.isscalar(P):
```

```
    P = np.eye(n)*P
```

```
else:
```

```
    P = np.atleast_2d(P)
```

```
lambda_ = self.alpha**2 * (n + self.kappa) - n
```

```
U = self.sqrt((lambda_ + n)*P)
```

```
sigmas = np.zeros((2*n+1, n))
```

```
sigmas[0] = x
```

```
for k in range(n):
```

```
    # pylint: disable=bad-whitespace
```

```
    sigmas[k+1] = self.subtract(x, -U[k])
```

```
    sigmas[n+k+1] = self.subtract(x, U[k])
```

```
return sigmas
```

```
def _compute_weights(self):
```

```
    """ Computes the weights for the scaled unscented Kalman filter. """
```

```
n = self.n
```

```
lambda_ = self.alpha**2 * (n + self.kappa) - n
```

```

c = .5 / (n + lambda_)
self.Wc = np.full(2*n + 1, c)
self.Wm = np.full(2*n + 1, c)
self.Wc[0] = lambda_ / (n + lambda_) + (1 - self.alpha**2 + self.beta)
self.Wm[0] = lambda_ / (n + lambda_)

```

```

def __repr__(self):
    return (f'MerweScaledSigmaPoints(n={self.n}, alpha={self.alpha}, '
            f'beta={self.beta}, kappa={self.kappa})')

```

```

def unscented_transform(sigmatas, Wm, Wc, noise_cov=None,
                        mean_fn=None, residual_fn=None):

```

```

    """

```

Computes unscented transform of a set of sigma points and weights.

returns the mean and covariance in a tuple.

:param residual\_fn:

:param sigmas: ndarray, of size (n, 2n+1)

2D array of sigma points.

:param Wm : ndarray [# sigmas per dimension]

Weights for the mean.

:param Wc : ndarray [# sigmas per dimension]

Weights for the covariance.

```

:param noise_cov: ndarray, optional
    noise matrix added to the final computed covariance matrix.

:param mean_fn: callable (sigma_points, weights), optional
    Function that computes the mean of the provided sigma points
    and weights
"""

kmax, n = sigmas.shape

try:
    if mean_fn is None:
        # new mean is just the sum of the sigmas * weight
        x = np.dot(Wm, sigmas) # dot = \Sigma^n_1 (W[k]*Xi[k])
    else:
        x = mean_fn(sigmas, Wm)
except:
    print(sigmas)
    raise

# new covariance is the sum of the outer product of the residuals
# times the weights

# this is the fast way to do this - see 'else' for the slow way
if residual_fn is np.subtract or residual_fn is None:
    y = sigmas - x[np.newaxis, :]
    P = np.dot(y.T, np.dot(np.diag(Wc), y))

```

else:

```
P = np.zeros((n, n))
```

```
for k in range(kmax):
```

```
    y = residual_fn(sigmas[k], x)
```

```
    P += Wc[k] * np.outer(y, y)
```

if noise\_cov is not None:

```
    P += noise_cov
```

```
return x, P
```

```
def Q_discrete_white_noise(dim, dt=1., var=1., block_size=1, order_by_dim=True):  
    """
```

Returns the Q matrix for the Discrete Constant White Noise

Model. dim may be either 2, 3, or 4 dt is the time step, and sigma  
is the variance in the noise.

Q is computed as the  $G * G^T * \text{variance}$ , where G is the process noise per  
time step. In other words,  $G = [[.5dt^2][dt]]^T$  for the constant velocity  
model.

:param dim : int (2, 3, or 4)

dimension for Q, where the final dimension is (dim x dim)

:param dt : float, default=1.0

time step in whatever units your filter is using for time. i.e. the  
amount of time between innovations

```
:param var : float, default=1.0
```

variance in the noise

```
:param block_size : int >= 1
```

If your state variable contains more than one dimension, such as a 3d constant velocity model  $[x \ x' \ y \ y' \ z \ z']^T$ , then Q must be a block diagonal matrix.

```
:param order_by_dim : bool, default=True
```

Defines ordering of variables in the state vector. ‘True‘ orders by keeping all derivatives of each dimensions)

$[x \ x' \ x'' \ y \ y' \ y'']$

whereas ‘False‘ interleaves the dimensions

$[x \ y \ z \ x' \ y' \ z' \ x'' \ y'' \ z'']$

''''''

```
if not (dim == 2 or dim == 3 or dim == 4):
```

```
    raise ValueError("dim must be between 2 and 4")
```

```
if dim == 2:
```

```
    Q = [[.25*dt**4, .5*dt**3],
          [.5*dt**3, dt**2]]
```

```
elif dim == 3:
```

```
    Q = [[.25*dt**4, .5*dt**3, .5*dt**2],
          [.5*dt**3, dt**2, dt],
          [.5*dt**2, dt, 1]]
```

```
else:
```

```
Q = [(dt**6)/36, (dt**5)/12, (dt**4)/6, (dt**3)/6],
      [(dt**5)/12, (dt**4)/4, (dt**3)/2, (dt**2)/2],
      [(dt**4)/6, (dt**3)/2, dt**2, dt],
      [(dt**3)/6, (dt**2)/2, dt, 1.]]
```

```
if order_by_dim:
```

```
    return block_diag(*[Q]*block_size) * var
```

```
return order_by_derivative(np.array(Q), dim, block_size) * var
```

```
def logpdf(x, mean=None, cov=1, allow_singular=True):
```

```
    """
```

Computes the log of the probability density function of the normal

N(mean, cov) for the data x. The normal may be univariate or multivariate.

```
    """
```

```
if mean is not None:
```

```
    flat_mean = np.asarray(mean).flatten()
```

```
else:
```

```
    flat_mean = None
```

```
flat_x = np.asarray(x).flatten()
```

```
return multivariate_normal.logpdf(flat_x, flat_mean,
```

```
                                   cov, allow_singular=allow_singular)
```

```
def fx_ballistic(x, dt):
```

```
    """
```

9D ballistic model (CV-CA-IMM): state = [x, vx, ax, y, vy, ay, z, vz, az].

Constant-acceleration discretisation:

$$p\_new = p + v*dt + 0.5*a\_total*dt^2$$

$$v\_new = v + a\_total*dt$$

a\_new = a (deterministic — it only changes through the Q[a] noise).

''''''

$$vx = np.clip(x[1], -40.0, 40.0)$$

$$vy = np.clip(x[4], -40.0, 40.0)$$

$$vz = np.clip(x[7], -40.0, 40.0)$$

$$ax\_s, ay\_s, az\_s = x[2], x[5], x[8]$$

$$k = 0.0314$$

$$v\_mag = np.sqrt(vx**2 + vy**2 + vz**2)$$

$$a\_drag\_x = -k * v\_mag * vx$$

$$a\_drag\_y = -k * v\_mag * vy$$

$$a\_drag\_z = -k * v\_mag * vz$$

$$g = 9.81$$

$$a\_tot\_x = ax\_s + a\_drag\_x$$

$$a\_tot\_y = ay\_s + a\_drag\_y - g$$

$$a\_tot\_z = az\_s + a\_drag\_z$$

```
return np.array([
    x[0] + vx*dt + 0.5*a_tot_x*dt**2,
    vx + a_tot_x*dt,
    ax_s,
    x[3] + vy*dt + 0.5*a_tot_y*dt**2,
```

```

vy + a_tot_y*dt,
ay_s,
x[6] + vz*dt + 0.5*a_tot_z*dt**2,
vz + a_tot_z*dt,
az_s,
])

```

```
def fx_hit(x, dt):
```

```

    """

```

9D hit model: structurally identical to fx\_ballistic (CA with gravity and drag). The difference is in the Q matrix (create\_imm\_estimator): the hit mode has a much larger Q[a], which lets the acceleration "wake up" from ~0 to impulsive values within a frame or two. A hit/block is thus remembered as a non-zero acceleration in the state, instead of the random velocity drift caused by a large Q[v] in the old 6D version.

```

    """

```

```

    return fx_ballistic(x, dt)

```

```
def fx_bounce(x, dt):
```

```

    """

```

9D floor-bounce model: vy is inverted with epsilon=0.75, vx/vz are damped with friction=0.85.

```

    """

```

```

    epsilon = 0.75

```

```

    friction = 0.85

```

```

    x_next = np.copy(x)

```



```

vx_new = x[1] * friction
vy_new = -x[4] * epsilon
vz_new = x[7] * friction

```

```

x_next[0] += vx_new * dt
x_next[3] += vy_new * dt
x_next[6] += vz_new * dt

```

```

x_next[1] = vx_new
x_next[4] = vy_new
x_next[7] = vz_new

```

# Reset residual acceleration — a bounce is a "fresh start".

```

x_next[2] = 0.0
x_next[5] = 0.0
x_next[8] = 0.0

```

```

return x_next

```

```

def hx(x):

```

```

    """

```

Measurement function 9D -> 3D: extract positions only (indices 0, 3, 6).

Velocities (1, 4, 7) and accelerations (2, 5, 8) are not measured, only estimated by the UKF/IMM.

```

    """

```

```

    return np.array([x[0], x[3], x[6]])

```

```
def measurement_transform(prediction_data, K, R_matrix, camera_pos,
                           ball_diameter=0.21):
```

```
    """
```

Turn a detection record (px, py, bbox width) into a 3D world measurement [X, Y, Z] in court coordinates, with the convention:

X — across the court (along the end line),  
 Y — vertical axis (up, perpendicular to the floor),  
 Z — along the long side (along the side line).

```
    """
```

```
    u_cam, v_cam, w_cam = (prediction_data['x_pos'], prediction_data['y_pos'],
                           prediction_data['w_box'])
```

```
    raw_x, raw_y, raw_z = get_3d_position(u_cam, v_cam, w_cam, K,
                                           R_matrix, camera_pos, ball_diameter)
```

```
    measurement = np.array([raw_x, raw_y, raw_z], dtype=np.float32)
```

```
    return measurement
```

```
def get_dynamic_transition_matrix(y, vy,
                                   mahalanobis_sq=0.0,
                                   hit_residual_min_sq=8.0,
                                   hit_residual_max_sq=11.34,
                                   bounce_height_max=0.55,
                                   frames_since_update=0,
                                   bounce_max_coast=3,
                                   m_hit_target=None):
```

```
    """
```

Return the 3x3 IMM Markov transition matrix for the models [Ballistic, Hit, Bounce] for the next prediction step.

The base matrix sticks to Ballistic ( $M[0,0]=0.95$ ): without external triggers the ball keeps following a ballistic trajectory.

Triggers:

1) Bounce — a passive geometric trigger.

2) Mid-air hit — a residual-based trigger.

Parameters:

- hit\_residual\_min\_sq=8.0: ~95th percentile of  $\chi^2_3$ ;

- hit\_residual\_max\_sq=11.34: upper bound = gating threshold.

Outside [min, max] the interpolation is clamped (0 below, 1 above).

:param y: ball vertical coordinate in world frame (m).

:param vy: vertical velocity (m/s).

:param mahalanobis\_sq: squared Mahalanobis distance of the last accepted detection against the IMM prediction (0 for tracks without updates).

:param hit\_residual\_min\_sq, hit\_residual\_max\_sq: activation range of the hit trigger (squared values matching the  $\chi^2_3$  distribution).

""""

```
M = np.array([[0.95, 0.04, 0.01],
              [0.60, 0.40, 0.00],
              [0.90, 0.00, 0.10]])
```

```
if frames_since_update > bounce_max_coast:
```

```
    return np.eye(3)
```

```
if y < bounce_height_max and vy < 0 and frames_since_update <= bounce_max_coast:
```

```
    M[0] = [0.10, 0.05, 0.85]
```

```
    M[1] = [0.10, 0.05, 0.85]
```

```
    return M
```

```
if mahalanobis_sq > hit_residual_min_sq:
```

```
    alpha = min(
```

```
        1.0,
```

```
        (mahalanobis_sq - hit_residual_min_sq)
```

```
        / max(hit_residual_max_sq - hit_residual_min_sq, 1e-6),
```

```
    )
```

```
    M_hit_target = (np.array([0.45, 0.50, 0.05])
```

```
        if m_hit_target is None
```

```
        else np.asarray(m_hit_target, dtype=float))
```

```
    M[0] = (1.0 - alpha) * M[0] + alpha * M_hit_target
```

```
    return M
```

### **ball\_3d\_analysis/IMMTracker.py**

```
import numpy as np
```

```
from scipy.optimize import linear_sum_assignment
```

```
from IMM_UKF import *
```

```
import cv2
```

```
def create_imm_estimator(z_initial, dt=0.02, v_initial=None):
```

```
    """
```

Створює IMM-естиматор з трьох UKF (ballistic / hit / bounce).

:param z\_initial: 3-вектор світової позиції м'яча [x, y, z] при ініціалізації.

:param dt: крок часу (= 1 / FPS).

:param v\_initial: опціональний 3-вектор початкових швидкостей [vx, vy, vz]

у м/с. Якщо None — швидкості ініціалізуються нулями.

Передавайте оцінку  $(z_k - z_{k-1}) / \Delta t$ , коли можемо

пов'язати поточну детекцію з найближчою попередньою

незасоційованою детекцією — це різко скорочує час

сходження UKF-фільтрів і запобігає катастрофічним

сплескам у  $|residual|/Mahalanobis$  у перші кадри

життя нового треку.

```
    """
```

```
# dt is forwarded from IMMTracker (1 / source FPS);
```

```
# default corresponds to 50 FPS.
```

```
# Step 2.B (Phase 3): 6D CV-state → 9D CA-state.
```

```
# state = [x, vx, ax, y, vy, ay, z, vz, az]
```

```
dim_x = 9
```

```
dim_z = 3
```

```
points = MerweScaledSigmaPoints(n=dim_x, alpha=.1, beta=2., kappa=1.)
```

```
# P_init для 9D стану. Накачуємо коваріацію accel ~25 (м/с²)²
```

```
# ( $\sigma_a \approx 5$  м/с²) — це дозволяє фільтру швидко поглинути перший
```

```
# ненульовий accel із вимірювань, не вибухнувши гейтом одразу.
```

```
# Якщо швидкість ініціалізована з фінітної різниці двох детекцій, її
```

```
#  $\sigma$  становить  $\approx \sqrt{2} \cdot \sigma_R / \Delta t$  (від ~7 м/с до ~14 м/с при дефолтних R, dt),
```

```

# тому коваріація швидкостей лишається 50.
P_init = np.diag([
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
    0.1, 50.0, 25.0,
])
# R: коваріація шуму вимірювання у світових координатах [x, y, z] (м²).
#  $\sigma_z \approx 0.7$  м (моно-камерна глибина має таку похибку через
#  $Z_c = f \cdot D / \text{bbox}_w$ ). Користувач підтвердив, що ці значення дають
# приємну плавність — спроба зменшити  $\sigma_z$  до 0.5 м (для зменшення
# відставання) робила траєкторію надто рваною.
R_init = np.diag([0.02, 0.02, 0.5])

# Q-матриці для 9D CA-state. У Q_discrete_white_noise(dim=3, ...)
# параметр var має семантику "дисперсія шуму джерку" — нижня
# права клітинка блоку = var, що задає, наскільки accel може
# випадково "крутитися" за крок.
#
# ballistic: var=0.1 — м'яч у вільному польоті фактично має accel = g
# + drag (smooth), залишковий accel-state  $\approx 0$ ; шум джерку маленький.
q_var_ballistic = 0.1
q_b = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_ballistic)
Q_ballistic = block_diag(q_b, q_b, q_b)

# hit: var=400 — це  $\sigma_{\text{jerk}} \approx 20 \text{ (м}^2/\text{с}^6)^{1/2}$ , що дозволяє accel-стейту
# "прокинутись" від  $\sim 0$  до десятків м/с² за один-два кадри. Тобто
# імпульсний удар реалізується НЕ через стрибок швидкості напряду,
# а через ріст accel-стейту, який потім інтегрується у швидкість.

```

```

# Це структурне моделювання маневру (CV-CA IMM Singer-style), на
# відміну від наївного збільшення  $Q[v]$ , що ламало гейтинг у 6D-версії.
q_var_hit = 400
q_h = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_hit)
Q_hit = block_diag(q_h, q_h, q_h)

# bounce: var=5 — помірний шум джерку. Після контакту з підлогою
# fx_bounce уже скидає accel у 0; Q[a] лише страхує від раптових
# відхилень у наступних кадрах.
q_var_bounce = 5
q_bnc = Q_discrete_white_noise(dim=3, dt=dt, var=q_var_bounce)
Q_bounce = block_diag(q_bnc, q_bnc, q_bnc)

# Ballistic filter
ukf_ballistic = UnscentedKalmanFilter(name='ballistic UKF', dim_x=dim_x,
                                       dim_z=dim_z, dt=dt, fx=fx_ballistic, hx=hx, points=points)
ukf_ballistic.P = P_init
ukf_ballistic.Q = Q_ballistic
ukf_ballistic.R = R_init

# Hit filter
ukf_hit = UnscentedKalmanFilter(name='hit UKF', dim_x=dim_x, dim_z=dim_z,
                                 dt=dt, fx=fx_hit, hx=hx, points=points)
ukf_hit.P = P_init
ukf_hit.Q = Q_hit
ukf_hit.R = R_init

# Bounce filter

```

```

ukf_bounce = UnscentedKalmanFilter(name='bounce UKF', dim_x=dim_x,
                                     dim_z=dim_z, dt=dt, fx=fx_bounce, hx=hx, points=points)
ukf_bounce.P = P_init
ukf_bounce.Q = Q_bounce
ukf_bounce.R = R_init

filters_lt = [ukf_ballistic, ukf_hit, ukf_bounce]
mu = np.array([0.95, 0.04, 0.01])
M_base = np.array([[0.95, 0.04, 0.01],
                    [0.60, 0.40, 0.00],
                    [0.90, 0.00, 0.10]])

imm = IMMEstimator(filters_lt, mu, M_base)

# Ініціалізуємо 9D стан [x, vx, ax, y, vy, ay, z, vz, az].
# Якщо передано v_initial — використовуємо його; інакше — нулі.
# accel-компоненти завжди стартують з 0: ми не знаємо, чи м'яч
# був у момент ініціалізації під дією impulse'y; пріор accel=0
# відповідає чистій балістиці. Високий P_init[a]=25 дозволяє
# фільтру швидко "виявити" ненульовий accel при потребі.
if v_initial is None:
    vx0 = vy0 = vz0 = 0.0
else:
    vx0, vy0, vz0 = (float(v_initial[0]), float(v_initial[1]),
                     float(v_initial[2]))
initial_x = np.array([
    z_initial[0], vx0, 0.0,
    z_initial[1], vy0, 0.0,

```



```

    z_initial[2], vz0, 0.0,
)
for f in imm.filters:
    f.x = initial_x.copy()
    # filterpy.IMMEstimator у конструкторі копіює f.x_post у imm.x_post
    # ОДИН РАЗ — і це відбувається ДО того, як ми перезаписали f.x.
    # Якщо не перевизначити f.x_post тут вручну, у першому snapshot треку
    # (на кадрі-народжування, ДО першого update'у) у JSONL потрапляє
    # zeros(6) → live_view/overlay малюють точку у світовому (0,0,0) =
    # лівий-передній кут майданчика → видима "лінія з кута" для кожного
    # нового треку. Поле x_post не впливає на predict/update — це лише
    # post-step snapshot для логування — тому правка безпечна.
    f.x_post = initial_x.copy()
imm._compute_state_estimate()
# Те саме на рівні IMM-агрегації: imm.x_post читається у snapshot_track
# з run_segment.py:214. Перевизначаємо тут до повернення.
imm.x_post = initial_x.copy()

return imm

class Track:
    # Санітарне обмеження на bootstrap-швидкість (м/с): якщо обчислена з
    # фінітної різниці швидкість перевищує цей поріг — bootstrap НЕ
    # застосовуємо, бо це майже напевно з'єднання двох різних об'єктів.
    BOOTSTRAP_MAX_SPEED = 35.0

    # Step 2.A (Phase 3): track-level covariance reset on radical residual.
    # Якщо поточний Mahalanobis^2 високий (хвіст ~5%  $\chi^2$ ), вважаємо, що

```

# м'яч щойно зробив імпульсний маневр (удар, блок) — і "відкриваємо"  
 # коваріацію швидкості та прискорення ПЕРЕД викликом imm.update(z).  
 # Більший P[v, a] = більший Kalman gain → новий вимір швидко  
 # "втягує" state у нову траєкторію без багатоканального лагу.  
 # Без цього: після удару фільтр бачить residual  $\approx 1$  м (z прискорився  
 # геометрично), повільно зменшує його за 3-5 кадрів, накопичує  
 # фантомні mah > гейту, і час від часу втрачає трек.  
 COV\_RESET\_MAH\_SQ\_THRESHOLD = 8.0 # ~95-й перцентиль  $\chi^2$   
 COV\_RESET\_INFLATE\_V = 50.0 # (м/с)<sup>2</sup> —  
 той самий рівень, що P\_init[v]  
 COV\_RESET\_INFLATE\_A = 100.0 # (м/с<sup>2</sup>)<sup>2</sup> —  
 4× P\_init[a], сильна "відкритість"

# Phase B: Delayed-accept hysteresis.  
 # Підозріла детекція (mah\_sq > THRESHOLD) не приймається одразу —  
 вона  
 # "заморожується" у \_pending і чекає підтвердження з наступних кадрів.  
 # Якщо підтвердження є (наступна детекція теж далеко) —  
 маневр реальний.  
 # Якщо наступна детекція повернулась до норми —  
 pending був FP, скидається.

HYSTERESIS\_MAH\_SQ\_THRESHOLD = 8.0 # поріг "підозри" (збігається з CO  
 HYSTERESIS\_CONFIRM\_THRESHOLD = 5.0 # mah\_sq наступної < цього → ре  
 HYSTERESIS\_WINDOW = 6 # максимум кадрів очікування підтверджен  
 COAST\_THRESHOLD = 4 # пропусків поспіль = "коастинг": висока  
 # mah\_sq очікувана (P зросла природно),  
 # а не симптом маневру — не inflate, не defer

# Coast covariance cap. Під час коастингу P росте необмежено (вимір.  
 # дані: P\_vel 2.5→700, P\_pos 0→243, P\_acc→1330 за 57 кадрів). При  
 # P\_vel~300-700 sigma-точки UKF розповзаються на ±17-26 м/с, і  
 # КВАДРАТИЧНИЙ drag ( $a = -k \cdot |v| \cdot v$  у fx\_ballistic) через нерівність Єнсена  
 # тягне sigma-mean швидкості до нуля, ПЕРЕБИВАЮЧИ лінійний gravity-term  
 # (-g) → v<sub>y</sub> замерзає (~-1 м/с) замість розгону вниз → трек «висить»  
 # замість балістичної дуги (tid4 завис на y=8 м усі 80 кадрів коастингу  
 # замість падіння під гравітацією). Кап тримає sigma-точки тісними →  
 # gravity знову домінує → чиста парабола поза кадром. Капи на рівні  
 # P\_init (vel) / cov-reset (acc); pos=9 (±3 м) узгоджено з Gate A  
 # residual-стелажем, тримає Mahalanobis-гейт осмисленим на коастингу.  
 COAST\_P\_POS\_CAP = 9.0 # (м<sup>2</sup>) — ±3 м, = Gate A max\_assoc\_residual  
 COAST\_P\_VEL\_CAP = 50.0 # (м/с)<sup>2</sup> — = P\_init[v]  
 COAST\_P\_ACC\_CAP = 100.0 # (м/с<sup>2</sup>)<sup>2</sup> — = COV\_RESET\_INFLATE\_A

```
def __init__(self, track_id, z_initial, dt, v_initial=None,
              initial_hits=1, min_hits=3, enable_hysteresis=False,
              enable_cov_reset=False, enable_coast_cov_cap=False,
              enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
              hit_residual_min_sq=8.0, hit_residual_max_sq=11.34,
              bounce_height_max=0.55, bounce_max_coast=3,
              m_hit_target=None):
```

"""

:param enable\_cov\_reset: вмикає Step 2.A track-level covariance reset  
 (inflate P[v,a] при mah\_sq>поріг). Default False  
 (Path B). Вимкнення лишає residual "пожити" 1-2  
 кадри після маневру → hit-тригер встигає  
 вистрілити і IMM-likelihood підіймає  $\mu_{hit}$ .

Увімкнений cov-reset снапить маневр за 1 кадр і короутить IMM-режими (cov-reset і hit — субститути за один і той самий residual-сигнал).

:param enable\_hysteresis: вмикає Phase B delayed-accept hysteresis

(відкладання підозрілих детекцій у \_pending).

Default False: на тест-даних гістерезис майже не вмикається, АЛЕ там, де вмикається, він канібалізує cov-reset (Step 2.A) та hit-тригер — усі три ділять поріг mah\_sq=8, і defer обнуляє \_last\_mahalanobis\_sq, тож наступний predict не бачить residual'у для hit-режиму. Тримаємо OFF, доки не доналаштуємо cov-reset+hit; coast-skip (через \_missed\_streak) лишається активним незалежно від прапорця — це обробка оклюзії, а не defer.

:param v\_initial: опціональний 3-вектор початкових швидкостей.

Якщо передано — UKF-фільтри стартують зі «справжнім» вектором швидкості замість нулів.

:param initial\_hits: к-сть hits, з якою стартує трек. Для

bootstrap'нутих треків розумно ставити 2 (бо ми фактично спостерігали дві послідовні детекції — попередню «незасоційовану» та поточну). Дозволяє швидше промувати трек до 'Confirmed', не чекаючи зайвого кадру.

:param min\_hits: к-сть оновлень (hits), необхідна для переходу

трек у стан 'Confirmed'. Раніше було захардкоджено 3 — тепер прокидається з IMMTracker (CLI), щоб параметр з sweep'а

справді впливав на трек-стейт-машину.

»»»»»

```
self.track_id = track_id
```

```
self.dt = float(dt)
```

```
self.enable_hysteresis = bool(enable_hysteresis)
```

```
self.enable_cov_reset = bool(enable_cov_reset)
```

```
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
```

```
# Coast Z-fence (2026-06-04): під час коастингу немає вимірювання, тож
```

```
# жоден гейт не діє — predict() екстраполює світову глибину Z (індекс 6)
```

```
# за контамінованою depth-швидкістю vz (індекс 7). На даних: vz dorостає
```

```
# до  $\pm 19$  м/с (бо біля межі видимої області  $w_{box}$  обвалюється  $\rightarrow Z=f \cdot D/w$ 
```

```
# роздувається), і коаст несе м'яч у  $Z=33-42$  м —
```

фізично неможливо на

```
# майданчику 18 м. Фенс клампить Z прогнозу у  $[coast\_z\_min, coast\_z\_max]$ 
```

```
# і гасить  $vz/az$ , які штовхають за фенс. Якщо м'яч справді покинув корт —
```

```
# max_age його видалить; доки коастить у межах — глибина лишається
```

```
# фізичною. Діє ЛИШЕ під час коасту (mark_missed), не чіпає update-кадри
```

```
self.enable_coast_z_fence = bool(enable_coast_z_fence)
```

```
self.coast_z_min = float(coast_z_min)
```

```
self.coast_z_max = float(coast_z_max)
```

```
# IMM-тригери (Phase F експеримент): прокидаються у
```

```
# get_dynamic_transition_matrix кожного predict(). Дефолти збігаються
```

```
# з сигнатурою функції — А/В-сумісність зі старими прогонами.
```

```
self.hit_residual_min_sq = float(hit_residual_min_sq)
```

```
self.hit_residual_max_sq = float(hit_residual_max_sq)
```

```
self.bounce_height_max = float(bounce_height_max)
```

```
self.bounce_max_coast = int(bounce_max_coast)
```

```

self.m_hit_target = (None if m_hit_target is None
                      else np.asarray(m_hit_target, dtype=float))
self.imm = create_imm_estimator(z_initial, dt, v_initial=v_initial)

# Запам'ятовуємо ПЕРШУ детекцію — використовується для
# внутрішньо-трекного bootstrap'у швидкості на другому хіті.
# Якщо v_initial вже передано ззовні (Mechanism B з IMMTracker),
# повторний bootstrap пропускаємо.
self.z_initial = np.asarray(z_initial, dtype=float).copy()
self._velocity_bootstrapped = (v_initial is not None)

# ЖИТТЄВИЙ ЦИКЛ
self.state = 'Tentative' # Можливі: 'Tentative', 'Confirmed', 'Deleted'
self.time_since_update = 0
self.hits = int(initial_hits)
self.hit_streak = int(initial_hits)
self.min_hits = int(min_hits)
# Останній прийнятий Mahalanobis^2 (gating-residual) — передається
# у get_dynamic_transition_matrix для активації hit-тригера у
# наступному predict(). Скидається у 0 після кадрів без оновлень.
self._last_mahalanobis_sq = 0.0
# Phase B: hysteresis state
self._pending = None # Optional[Tuple[np.ndarray, float]]: (z, mah_sq)
self._pending_frames_elapsed = 0 # кадрів з моменту встановлення pending
self._missed_streak = 0 # послідовних mark_missed (для coast detection)
# 9D h_matrix: позиційні компоненти — індекси 0, 3, 6 у state
# [x, vx, ax, y, vy, ay, z, vz, az]. Швидкості та прискорення
# не вимірюються напряму, лише оцінюються через UKF.

```

```
self.h_matrix = np.array([
    [1, 0, 0, 0, 0, 0, 0, 0, 0], # x
    [0, 0, 0, 1, 0, 0, 0, 0, 0], # y
    [0, 0, 0, 0, 0, 0, 1, 0, 0], # z
], dtype=float)
```

```
def _inflate_covariance(self):
```

```
    """Inflate P[v,a] для всіх sub-фільтрів та IMM-агрегації."""
```

```
    for f in self.imm.filters:
```

```
        f.P[1, 1] += self.COV_RESET_INFLATE_V
```

```
        f.P[4, 4] += self.COV_RESET_INFLATE_V
```

```
        f.P[7, 7] += self.COV_RESET_INFLATE_V
```

```
        f.P[2, 2] += self.COV_RESET_INFLATE_A
```

```
        f.P[5, 5] += self.COV_RESET_INFLATE_A
```

```
        f.P[8, 8] += self.COV_RESET_INFLATE_A
```

```
    self.imm.P[1, 1] += self.COV_RESET_INFLATE_V
```

```
    self.imm.P[4, 4] += self.COV_RESET_INFLATE_V
```

```
    self.imm.P[7, 7] += self.COV_RESET_INFLATE_V
```

```
    self.imm.P[2, 2] += self.COV_RESET_INFLATE_A
```

```
    self.imm.P[5, 5] += self.COV_RESET_INFLATE_A
```

```
    self.imm.P[8, 8] += self.COV_RESET_INFLATE_A
```

```
def _collapse_accel_to_ballistic(self):
```

```
    """Занулює залишкове прискорення (індекси 2, 5, 8) у всіх станах.
```

Викликається при коастингу (mark\_missed). Залишковий accel у 9D

СА-стані валідний лише поки його підтверджують виміри: без виміру

predict() інтегрує застаріле ax у швидкість щокадру (vx += ax\*dt) →

трек РОЗГАНЯЄТЬСЯ і летить через увесь корт (аномалія кадрів 611-650:  $v_X$  тікав  $-3.2 \rightarrow -9.7$  за 33 кадри оклюзії,  $X$   $7.2 \rightarrow 2.4$ ).

Фізично м'яч між контактами має нульовий залишковий accel — лише  $g + drag$ , які вже driving-terms у  $fx\_*$ . Колапс до балістики лишає коастинг constant-velocity замість нестабільної СА-екстраполяції. Швидкість/позиція не чіпаються (вони фізично виправдані),  $P$  також.  
”””””

```
for f in self.imm.filters:
```

```
    f.x[2] = f.x[5] = f.x[8] = 0.0
```

```
    f.x_post[2] = f.x_post[5] = f.x_post[8] = 0.0
```

```
self.imm.x[2] = self.imm.x[5] = self.imm.x[8] = 0.0
```

```
self.imm.x_post[2] = self.imm.x_post[5] = self.imm.x_post[8] = 0.0
```

@staticmethod

```
def _cap_P_matrix(P, caps):
```

```
    """PD-безпечний кап діагоналі P до caps[i]. Для кожного i з
```

```
    P[i,i] > caps[i] масштабуємо рядок і стовпець i на  $s=\sqrt{cap/P[i,i]}$ .
```

```
    Це конгруентне перетворення  $D \cdot P \cdot D$  ( $D=diag, d_i=s$ ), що ЗБЕРІГАЄ  
    додатну визначеність (на відміну від наївного  $P[i,i]=cap$ , яке при  
    великих позадіагональних кореляціях ламає  $PD \rightarrow Cholesky$ -збій у  
    sigma-точках UKF). Off-діагоналі масштабуються пропорційно, тож  
    кореляційна структура лишається валідною.”””””
```

```
    for i in range(P.shape[0]):
```

```
        if P[i, i] > caps[i]:
```

```
            s = np.sqrt(caps[i] / P[i, i])
```

```
            P[i, :] *= s
```

```
            P[:, i] *= s
```



```
def _cap_coast_covariance(self):
```

```
    """Обмежує ріст P-діагоналі під час коастингу (див. COAST_P_*_CAP).
```

```
    Без капю P росте необмежено → sigma-точки UKF розповзаються →  
    квадратичний drag глушить gravity у sigma-mean → vy замерзає, трек  
    «висить» замість балістичної дуги. Кап тримає sigma тісними →  
    чиста парабола. Доповнює Gate A (обидва б'ють runaway P)."""
```

```
    caps = np.array([
```

```
        self.COAST_P_POS_CAP, self.COAST_P_VEL_CAP, self.COAST_P_ACC_C
```

```
        self.COAST_P_POS_CAP, self.COAST_P_VEL_CAP, self.COAST_P_ACC_C
```

```
        self.COAST_P_POS_CAP, self.COAST_P_VEL_CAP, self.COAST_P_ACC_C
```

```
    ])
```

```
    for f in self.imm.filters:
```

```
        self._cap_P_matrix(f.P, caps)
```

```
    self._cap_P_matrix(self.imm.P, caps)
```

```
def _fence_coast_depth(self):
```

```
    """Клампить світову глибину Z (індекс 6) під час коасту в межі
```

```
    [coast_z_min, coast_z_max] і гасить depth-швидкість/прискорення
```

```
    (vz=7, az=8), що штовхають за фенс. Застосовується до ВСІХ внутрішніх
```

```
    UKF-фільтрів (їхні x та x_prior — основа sigma-точок наступного
```

```
    predict) та до mixed-стану imm (репортиться в overlay). Без цього
```

```
    контамінована vz екстраполює Z до 30-42 м (див. enable_coast_z_fence).
```

```
    P не чіпаємо — за нього відповідає Gate C (coast_cov_cap)."""
```

```
    zlo, zhi = self.coast_z_min, self.coast_z_max
```

```
def _clamp(x):
```

```
    if x[6] > zhi:
```

```
        x[6] = zhi
```

```

        if x[7] > 0.0:
            x[7] = 0.0
            x[8] = 0.0
        elif x[6] < zlo:
            x[6] = zlo
            if x[7] < 0.0:
                x[7] = 0.0
                x[8] = 0.0

    for f in self.imm.filters:
        _clamp(f.x)
        _clamp(f.x_prior)
    _clamp(self.imm.x)
    _clamp(self.imm.x_prior)

def predict(self):
    # 9D state: y = x[3], vy = x[4] (было 6D: x[2], x[3]).
    self.imm.M = get_dynamic_transition_matrix(
        self.imm.x[3], self.imm.x[4],
        mahalanobis_sq=self._last_mahalanobis_sq,
        frames_since_update=self.time_since_update,
        hit_residual_min_sq=self.hit_residual_min_sq,
        hit_residual_max_sq=self.hit_residual_max_sq,
        bounce_height_max=self.bounce_height_max,
        bounce_max_coast=self.bounce_max_coast,
        m_hit_target=self.m_hit_target,
    )
    self.imm.predict()

```

if self.\_pending is not None:

self.\_pending\_frames\_elapsed += 1

if self.\_pending\_frames\_elapsed > self.HYSTERESIS\_WINDOW:

# Вікно підтвердження вичерпане — скидаємо pending і

# відновлюємо нормальне старіння.

self.\_pending = None

self.\_pending\_frames\_elapsed = 0

self.time\_since\_update += 1

# else: трек не старіє під час активного вікна hysteresis

else:

self.time\_since\_update += 1

def update(self, z, R=None):

# R — опційна per-detection коваріація вимірювання (адаптивний  $\sigma_Z$ ).

# None → IMM-фільтри беруть власний фіксований self.R.

current\_mah\_sq = self.\_last\_mahalanobis\_sq

# — Phase B: Delayed-accept hysteresis —

# Defer-механізм (pending) увімкнено лише при enable\_hysteresis.

# Coast-skip (через \_missed\_streak) лишається активним завжди — це

# обробка повернення з оклюзії, а не відкладання детекції.

\_coast\_skip = False # True → пропустити inflate та hysteresis-тригер

if self.enable\_hysteresis and self.\_pending is not None:

pending\_z, \_ = self.\_pending

self.\_pending = None

self.\_pending\_frames\_elapsed = 0

```

if current_mah_sq >= self.HYSTERESIS_CONFIRM_THRESHOLD:
    # Обидва кадри далеко від прогнозу → реальний маневр підтверджено.
    # Inflate P + update з поточним z.
    # pending_z (пізній вимір) не застосовуємо: подвійний imm.update()
    # без predict() між ними порушує PD-властивість P → Cholesky fail.
    # Inflate + поточний z достатньо для фіксації нової траєкторії.
    self._inflate_covariance()
    self.imm.update(z, R=R)
    self.time_since_update = 0
    self.hits += 1
    self.hit_streak += 1
    if self.state == 'Tentative' and self.hits >= self.min_hits:
        self.state = 'Confirmed'
    self._missed_streak = 0
    return

# mah_sq < CONFIRM_THRESHOLD → поточна детекція повернулась до
# нормальної траєкторії → pending був false-positive.
# Продовжуємо нормальним update без inflate та без нового тригера.
_coast_skip = True
self._missed_streak = 0

elif self._missed_streak >= self.COAST_THRESHOLD:
    # Трек довго не бачив м'яча → висока mah_sq є ОЧІКУВАНОЮ
    # (коваріація P зросла природно під час коастингу), а не
    # ознакою маневру. Пропускаємо inflate і hysteresis-тригер.
    _coast_skip = True
    self._missed_streak = 0

```

```

elif (self.enable_hysteresis and z is not None
      and current_mah_sq > self.HYSTERESIS_MAH_SQ_THRESHOLD):
    # Підозріла детекція — відкладаємо її на підтвердження.
    self._pending = (np.asarray(z, dtype=float).copy(), current_mah_sq)
    self._pending_frames_elapsed = 0
    self.imm.update(None) # екстраполяція замість прийняття
    self.hit_streak = 0
    self._last_mahalanobis_sq = 0.0
    self._missed_streak += 1
    return

# — End Phase B —

# Mechanism A — Intra-track velocity bootstrap.
# На переході hits=1 → 2 (тобто коли трек уперше отримує
# ДРУГУ детекцію після створення) UKF-фільтри ще мали v=0 у
# пріорі поточного кадру. Це призводить до:
# 1) несхожої позиції-прогнозу (за реальної v>0 ball дрейфує
# від прогнозу) → велике |residual|, велика Mahalanobis;
# 2) повільного «зсуву» Kalman-gain'ом швидкості до правди
# (через велику  $\sigma_v = \sqrt{50} \approx 7$  м/с,  $K_v$  нікчемно мала за 1 крок).
# Виправляємо це примусово: ОБРАХОВУЄМО v_bootstrap з фінітної
# різниці  $(z - z_{\text{initial}}) / (\text{gap} \cdot dt)$  і ВПИСУЄМО її у швидкісні
# компоненти кожного фільтра до того, як викличемо imm.update.
# Так predict-update проходять із коректним пріором швидкості з
# САМОГО ПЕРШОГО використання, а Mahalanobis на 3-му кадрі вже
# дорівнює 0–1 замість 3–4.
do_bootstrap = (

```

```

z is not None
and self.hits == 1
and not self._velocity_bootstrapped
and self.time_since_update > 0
)
if do_bootstrap:
    gap_dt = self.time_since_update * self.dt
    z_arr = np.asarray(z, dtype=float)
    v_boot = (z_arr - self.z_initial) / gap_dt
    speed = float(np.linalg.norm(v_boot))
    if 0.0 < speed <= self.BOOTSTRAP_MAX_SPEED:
        # Вписуємо швидкість у ВСІ внутрішні UKF-фільтри (вони
        # роздільні; mixed-стан перерахується автоматично у
        # imm.update → _compute_state_estimate).
        # Так само переписуємо x_prior, бо саме він
        # використовуватиметься у sigma-точках наступного
        # predict.
        # 9D state: швидкісні індекси [1, 4, 7] (було 6D: [1, 3, 5]).
        # Accel-індекси (2, 5, 8) НЕ чіпаємо — лишаються 0 з
        # create_imm_estimator (немає підстав bootstrap'ити).
        for f in self.imm.filters:
            f.x[1] = v_boot[0]
            f.x[4] = v_boot[1]
            f.x[7] = v_boot[2]
            f.x_prior[1] = v_boot[0]
            f.x_prior[4] = v_boot[1]
            f.x_prior[7] = v_boot[2]
        # mixed state теж оновлюємо.

```

```

self.imm.x[1] = v_boot[0]
self.imm.x[4] = v_boot[1]
self.imm.x[7] = v_boot[2]
self.imm.x_prior[1] = v_boot[0]
self.imm.x_prior[4] = v_boot[1]
self.imm.x_prior[7] = v_boot[2]
self._velocity_bootstrapped = True

# Step 2.A — Track-level covariance reset on radical residual.
# Виконується ПІСЛЯ Mechanism A bootstrap (щоб не конфліктувати:
# у bootstrap'ному кадрі м'яч ще не "робив" удар, він просто
# підхопив реальну швидкість), але ПЕРЕД imm.update(z).
# _coast_skip = True означає, що висока mah_sq є очікуваною
# (коастинг або resolved FP pending) — inflate не потрібен.
do_cov_reset = (
    self.enable_cov_reset
    and z is not None
    and not _coast_skip
    and self._last_mahalanobis_sq > self.COV_RESET_MAH_SQ_THRESHOLD
)
if do_cov_reset:
    self._inflate_covariance()

self.imm.update(z, R=R)
self.time_since_update = 0
self.hits += 1
self.hit_streak += 1
if self.state == 'Tentative' and self.hits >= self.min_hits:

```

```

        self.state = 'Confirmed'

        self._missed_streak = 0

def mark_missed(self):
    self.hit_streak = 0

    # На кадрах без вимірювання residual не визначений — скидаємо у 0,
    # щоб у наступному predict() hit-тригер не активувався фантомно
    # з застарілих даних.
    self._last_mahalanobis_sq = 0.0
    self._missed_streak += 1 # Phase B: відстежуємо серію пропусків
    self.imm.update(None)    # Екстраполяція без вимірювання
    # Фікс кадрів 611-650: гасимо залишковий accel, щоб коастинг
    # не розганявся через інтеграцію застарілого ах у швидкість.
    self._collapse_accel_to_ballistic()
    # Gate C: кап росту P-діагоналі під час коастингу. Без нього P_vel
    # розбухає (2.5→700 за ~57 кадрів), сигма-точки UKF розкидаються
    # ±17-26 м/с, квадратичний drag (Jensen) тягне середню швидкість до
    # нуля й перебиває гравітацію → ву «замерзає», м'яч зависає замість
    # балістичної дуги. Кап тримає екстраполяцію фізичною за межами кадру.
    if self.enable_coast_cov_cap:
        self._cap_coast_covariance()
    # Coast Z-fence: тримаємо екстрапольовану глибину фізичною (≤ майданчик).
    if self.enable_coast_z_fence:
        self._fence_coast_depth()

def get_mahalanobis_distance(self, z, R_matrix):
    """
    Обчислює відстань Махаланобіса між прогнозом IMM та

```



НОВОЮ детекцією.

»»»»»

```
# Проектуємо змішаний стан x_prior та коваріацію
```

```
# P_prior у простір вимірювань
```

```
z_mean = np.dot(self.h_matrix, self.imm.x_prior)
```

```
S = np.dot(self.h_matrix, np.dot(self.imm.P_prior,  
                                self.h_matrix.T)) + R_matrix
```

```
y = z - z_mean # Вектор невязки
```

```
try:
```

```
    S_inv = np.linalg.inv(S)
```

```
    dist_sq = np.dot(y.T, np.dot(S_inv, y))
```

```
    return dist_sq
```

```
except np.linalg.LinAlgError:
```

```
    return 1e5
```

```
class IMMTracker:
```

```
    def __init__(self, dt=0.02, max_age=80, min_hits=3, gating_threshold=16.0,
```

```
                bootstrap_max_gap_frames=5, bootstrap_max_dist=2.5,
```

```
                bootstrap_max_speed=35.0, enable_hysteresis=False,
```

```
                enable_cov_reset=False, enable_single_ball_nms=True,
```

```
                enable_physical_gate=True, max_assoc_residual=3.0,
```

```
                enable_coast_cov_cap=False, spawn_suppress_max_coast=0,
```

```
                enable_depth_robust_gate=False, depth_hold_gain=0.5,
```

```
                depth_innov_free=0.0,
```

```
                enable_coast_z_fence=False, coast_z_min=-3.0, coast_z_max=21.0,
```

tentative\_max\_age=5, floor\_kill\_y=-0.3,  
 enable\_adaptive\_depth\_R=False,  
 hit\_residual\_min\_sq=8.0, hit\_residual\_max\_sq=11.34,  
 bounce\_height\_max=0.55, bounce\_max\_coast=3,  
 m\_hit\_target=None):

»»»»»

:param dt:                    крок часу (= 1 / FPS).

:param max\_age:                к-сть кадрів без оновлення до  
 видалення треку.

Історія: 150 (sweep) → 15 (Plan A) →  
 80 (Phase B + edge-gate, чиста  
 розгортка).

Старий 150 давав 3-секундний  
 коастинг — підтверджений трек, що  
 втратив детекцію, балістично летів  
 через увесь кадр і накопичував  
 ”фантомні” підтверджені треки на  
 сміттєвих детекціях. Plan A зрізав до  
 15, щоб їх убити. Але edge-gate  
 (court-bounds у run\_segment) тепер  
 прибирає самі викиди в джерелі, тож  
 головна причина для 15 зникла.

Розгортка з гейтом: 80 дає найкращу  
 fragmentation (0.235) + coverage  
 (0.907) при mode-switch 2.54 Гц  
 (нижче історичної бази ~3.5);  
 повний ~13-сек трек відновлюється.  
 100 доміновано 80-кою.

:param min\_hits: к-сть кадрів для підтвердження треку (Tentative → Confirmed).

:param gating\_threshold:  $\chi^2$ -поріг гейтингу Махаланобіса.  
 $\chi^2 \square p=0.99 = 11.34$ ; default 16.0  
пускає прикордонні детекції маневру (mah 12-15) асоціюватись → оживляє hit-режим без зростання проліферації.

:param bootstrap\_max\_gap\_frames: максимальний розрив (кадри) між пендінговою детекцією та поточною, щоб виконати bootstrap швидкості.  
Default 5 (= 0.1 с при 50 FPS).

:param bootstrap\_max\_dist: максимальна 3D-відстань (м) між пендінговою та поточною детекцією для bootstrap. Default 2.5 м — пасує для м'яча, що рухається до ~25 м/с, при розривах  $\leq 0.1$  с.

:param bootstrap\_max\_speed: санітарне обмеження на bootstrap-швидкість (м/с). Якщо обчислена  $|v| >$  цього порогу — не використовуємо bootstrap (швидше за все фальшиве з'єднання двох різних об'єктів). Default 35 м/с — приблизна верхня межа подачі елітного м'яча у волейболі.

:param enable\_hysteresis: вмикає Phase B delayed-accept hysteresis у кожному Track. Default False (див. Track.\_\_init\_\_ — defer канібалізує cov-reset+hit). Прокидається

з run\_segment через --enable\_hysteresis.

:param enable\_cov\_reset: вмикає Step 2.А covariance reset у кожному Track. Default False (Path B), щоб оживити IMM hit-режим. run\_segment --enable\_cov\_reset повертає назад.

:param enable\_single\_ball\_nms: Phase C. Домен — один м'яч, тож одночасно має бути лише ОДИН Confirmed-трек. Якщо їх >1 (фантом-дублі від сміттєвих спавнів / дивергентного коастингу), лишаємо найкращий, решту понижуємо до Tentative. Default True; run\_segment --disable\_single\_ball\_nms.

:param enable\_physical\_gate: Gate A. Коваріаційно-незалежний стелаж на евклідів residual  $\|z - \text{прогноз}\|$ .

Mahalanobis-гейт залежить від P, а P необмежено роздувається під час коастингу (tsu=47 → P\_pos ~80-130) → стрибок-FP на 8 м дає mah≈0.7 < гейт і ПРИЙМАЄТЬСЯ (катастрофа fr763/807/86: трек телепортує на сміттєву детекцію). Фіксований стелаж на residual незалежний від коастингу й ловить fr763 (resid 7.95) так само, як короткий коастинг ловив fr867 (mah=33). Default True; run\_segment --disable\_physical\_gate.

:param max\_assoc\_residual: поріг Gate A (м). Розподіл прийнятих апдейтів: p99=7.95, але реальний-маневр максимум ≈2.2 м (fr283 resid 2.19,

mah 13; подача fr650 resid 2.08); чиста  
 прірва 2.2 → 5.95 м до найближчого  
 телепорта. 3.0 м: +0.8 м над легіт-  
 максимумом, −3 м під телепортом.

»»»»

```

self.dt = dt
self.max_age = max_age
self.min_hits = min_hits
self.gating_threshold = gating_threshold

self.bootstrap_max_gap_frames = int(bootstrap_max_gap_frames)
self.bootstrap_max_dist = float(bootstrap_max_dist)
self.bootstrap_max_speed = float(bootstrap_max_speed)
self.enable_hysteresis = bool(enable_hysteresis)
self.enable_cov_reset = bool(enable_cov_reset)
self.enable_single_ball_nms = bool(enable_single_ball_nms)
# Gate A: коваріаційно-незалежний фізичний стелаж на residual.
self.enable_physical_gate = bool(enable_physical_gate)
self.max_assoc_residual = float(max_assoc_residual)
# Gate C: кап росту P-діагоналі під час коастингу (передається у Track).
self.enable_coast_cov_cap = bool(enable_coast_cov_cap)
# Gate D: придушення спавну паразитного треку, поки реальний Confirmed-
# трек КОРОТКО коастить. Домен — один м'яч. Коли м'яч на 1-2 кадри
# зникає (руки гравця, злиплий/стиснутий bbox), його w колапсує →
# implied Z стрибає на 5-11 м → Gate A (правильно) робить реальний трек
# коастуючим. АЛЕ та сама стрибуча детекція раніше спавнила КОНКУРЕНТА
# який за min_hits кадрів ставав Confirmed і через single-ball NMS
# (tsu_tol=1) ПЕРЕХОПЛЮВАВ ідентичність у 48-хітового реального треку —
    
```

|     |      |          |       |      |                    |       |
|-----|------|----------|-------|------|--------------------|-------|
|     |      |          |       |      | ІАЛЦ.467200.007 Д4 | Аркуш |
| Зм. | Лист | № докум. | Підп. | Дата |                    | 60    |

# filtZ телепортувало 6→16 м, ще й паразитна vz з bootstrap. Цей гейт  
 # НЕ дає народитись конкуренту, поки існує живий Confirmed-трек із  
 #  $0 < tsu \leq \text{spawn\_suppress\_max\_coast}$ : викид просто ігнорується, а  
 # коастуючий трек ре-захоплює м'яч, щойно повернеться валідна (Gate A)  
 # детекція. Після цього порогу коасту м'яч вважається СПРАВДІ  
 # переміщенням → спавн знову дозволено (легітимний handoff після довгої  
 # втрати). 0 = вимкнено (A/B-сумісність). Рек. ~8 (вкриває типову  
 # оклюзію руками  $\leq 8$  кадрів при 50 FPS).  
 self.spawn\_suppress\_max\_coast = int(spawn\_suppress\_max\_coast)  
 # Gate A-depth: depth-robust Gate A. Розкладаємо residual на ЛАТЕРАЛЬ  
 # ( $X=\text{idx0}$  ширина,  $Y=\text{idx1}$  висота — це проєкція пікселя u,v, яку YOLO дає  
 # коректно навіть при оклюзії) та ГЛИБИНУ ( $Z=\text{idx2}$  — монокулярна  
 #  $f \cdot D/w$ , що деградує, коли w колапсує під час оклюзії руками).  
 # При оклюзії residual латералі лишається  $< 1$  м, а residual глибини  
 # стрибає на 4-11 м. Звичайний Gate A ( $\|z\text{-прогноз}\|$ ) відкидає таку  
 # детекцію цілком → реальний трек коастить, ВТРАЧАЮЧИ хороше u,v  
 # (траєкторія відстає від м'яча — скарга користувача на Gate D).  
 # depth-robust замість цього: (1) гейтить ЛИШЕ латераль жорстко  
 # ( $\text{hypot}(dX, dY) \leq \text{max\_assoc\_residual}$ ) — детекцію з правильним u,v  
 # ПРИЙМАЄМО; (2) велику інновацію глибини НЕ відкидаємо, а сильно  
 # НЕДОВІРЯЄМО: роздуваємо  $\sigma_Z^2$  на  $(\text{depth\_hold\_gain} \cdot \text{depth\_innov})^2$  і в  
 # гейтингу (Mahalanobis), і на update-кроці. Глибину тоді тримає  
 # прогноз (без телепорту 16 м), а латераль X,Y оновлюється з валідної  
 # детекції (траєкторія приклеєна до м'яча, як у overlay\_D027). Реальний  
 # трек з'їдає детекцію → фантом не спавниться → Gate D непотрібен.  
 # Default False (A/B-сумісність). depth\_hold\_gain $\approx 0.5$ : при innov=8 м  
 #  $\sigma_Z\text{-eff} \approx 4$  м → детекція майже не зсуває depth, прогноз домінує.

```

self.enable_depth_robust_gate = bool(enable_depth_robust_gate)
self.depth_hold_gain = float(depth_hold_gain)
# Мертва зона (Huber-style) для depth-distrust: демпфуюмо лише
# НАДЛИШОК |innov| понад цей поріг нормальної фізики. excess =
# max(0, |innov| - depth_innov_free);  $\sigma_{Z^2} += (\text{gain} \cdot \text{excess})^2$ .
# МОТИВАЦІЯ: чисте  $(\text{gain} \cdot \text{innov})^2$  душило і легітимну глибину —
# на дальній стороні корту (rawZ природно шумить  $\pm 1.5\text{-}2$  м  $\rightarrow$ 
# filtZ залипав занизько) та на подачі (innov  $-4.5$  м  $\rightarrow$  filtZ повз
# 12 кадрів замість швидкої збіжності). Мертва зона  $\sim 3$  м пускає
# нормальну фізику без демпфування, а давить лише викиди оклюзії
# (innov 6-11 м). 0.0 = вимкнено (чисте квадратичне демпфування,
# A/B-сумісність). Рек.  $\sim 3.0$  (= max_assoc_residual: «нормальний»
# фізичний крок).
self.depth_innov_free = float(depth_innov_free)
# Coast Z-fence (2026-06-04): під час коасту немає вимірювання  $\rightarrow$  жоден
# гейт не діє, а predict() екстраполює світову Z за контамінованою vz
# (біля межі кадру w_box обвалюється  $\rightarrow Z=f \cdot D/w$  роздувається  $\rightarrow vz \pm 19$  м/с).
# На даних коаст ніс м'яч у  $Z=33\text{-}42$  м (майданчик лише 18 м). Фенс у
# mark_missed кожного треку клампить Z прогнозу в [coast_z_min,
# coast_z_max] і гасить vz/az за фенсом. Default False (A/B-сумісність).
# Рек. coast_z_max  $\approx 21$  (= back line 18 м +  $\sim 3$  м на сервера за лінією),
# coast_z_min  $\approx -3$  (подача з-за ближньої лінії). Доповнює far-Z стелю
# детекції (--court_z_far_max) у run_segment: одна давить джерело
# (контамінований вимір), друга — екстраполяцію.
self.enable_coast_z_fence = bool(enable_coast_z_fence)
self.coast_z_min = float(coast_z_min)
self.coast_z_max = float(coast_z_max)
# Дефект 1: привид-треки. Tentative-трек, що не дійшов до min_hits,

```

```

# не повинен коастити як повноцінний — 1-2 шумові детекції інакше
# породжують ~80-кадрову фікцію (tid3/5/7: 0 confirmed, 99% coast,
# під підлогою). Вбиваємо Tentative після цього к-ва коаст-кадрів.
self.tentative_max_age = int(tentative_max_age)

# Дефект 2: підпідлоговий коаст. М'яч у польоті не може бути нижче
# підлоги; якщо трек НА КОАСТІ екстраполює  $y < \text{floor\_kill\_y}$  —
# екстраполяція стала фікцією (tid2 пірнав до  $-5.6$  м), термінуємо.
# Невеликий від'ємний запас проти шуму біля підлоги (м'яч  $r \approx 0.1$  м).
self.floor_kill_y = float(floor_kill_y)

# Phase F: адаптивний  $\sigma_Z$ . Якщо True й у update() передано
# detection_covs — гейтинг і IMM-апдейт використовують per-detection
#  $3 \times 3$  R (анізотропний, витягнутий уздовж променя;  $\sigma_{Zc} = f \cdot D / w^2 \cdot \sigma_w$ ),
# а не фіксований self.R_matrix. Чесна довіра до глибини залежно від
# ширини bbox. Default False (A/B-сумісність).
self.enable_adaptive_depth_R = bool(enable_adaptive_depth_R)

# IMM-тригери (Phase F експеримент): прокидаються у кожен Track при
# створенні  $\rightarrow$  get_dynamic_transition_matrix. Дозволяють тюнити
# активацію bounce/hit-режимів з CLI без правок коду. Дефолти =
# сигнатура get_dynamic_transition_matrix (A/B-сумісність).
self.hit_residual_min_sq = float(hit_residual_min_sq)
self.hit_residual_max_sq = float(hit_residual_max_sq)
self.bounce_height_max = float(bounce_height_max)
self.bounce_max_coast = int(bounce_max_coast)
self.m_hit_target = (None if m_hit_target is None
                      else np.asarray(m_hit_target, dtype=float))

# Gate B: merge-identity guard. Два Confirmed-треки успадковують
# спільний id лише якщо фізично примиренні (той самий м'яч,
# роздроблений маневром:  $\leq$  цієї відстані). Далекий FP-трек, що

```



```

# дожив до Confirmed, НЕ донорить id реальному коастуючому треку
# (інакше merge-identity ампліфікувала б телепорт). Маневр-фрагменти
#  $\leq \sim 2$  м, FP-телепорти  $\geq 6$  м  $\rightarrow 4.0$  м чисто розділяє.
self.SINGLE_BALL_MERGE_MAX_DIST = 4.0

# Phase C: коастинг  $\leq$  цієї к-сті кадрів не коштує ідентичності
# (bucket'имо tsu). tol=1: реальний трек переживає ОДИН пропущений
# кадр (детекційний шум), але після 2+ кадрів коастингу поступається
# треку, що активно отримує детекції. Маленький tol важливий: під
# час маневру програваний трек коастить по старій траєкторії й
# ВІДХОДИТЬ від м'яча — чим раніше перемкнутись на детектований
# трек, тим менший просторовий стрибок на switch'і.
self.SINGLE_BALL_NMS_TSU_TOL = 1

self.tracks = []
self.next_id = 1

# Матриця для гейтингу Махаланобіса. Узгоджуємо з R_init у
# create_imm_estimator ( $\sigma_z \approx 0.7$  м для моно-камерної глибини).
self.R_matrix = np.diag([0.02, 0.02, 0.5])

# Ring-буфер невідповідних (unassigned) детекцій з кількох останніх
# кадрів. Використовується для bootstrap'у швидкості при створенні
# нового треку: якщо поточна незасоційована детекція близька до
# детекції з останніх N кадрів, ініціалізуємо новий трек з
#  $v = (z_{\text{now}} - z_{\text{prev}}) / (\Delta fr \cdot dt)$  замість нулів.
self._spawn_buffer = [] # елементи: (np.array z, int frame_idx)
self._frame_counter = 0

def update(self, detections_3d, detection_covs=None):

```

"""

detections\_3d: список масивів np.array([x, y, z]) для

всіх знайдених об'єктів у кадрі

detection\_covs: опційний паралельний список 3×3 коваріацій

вимірювання (по одній на детекцію). Вживається лише якщо

enable\_adaptive\_depth\_R=True. None → фіксований self.R\_matrix.

"""

self.\_frame\_counter += 1

def \_R\_for(d\_idx):

"""R для d-ї детекції: адаптивна, якщо ввімкнено й надано."""

if (self.enable\_adaptive\_depth\_R

and detection\_covs is not None

and detection\_covs[d\_idx] is not None):

return detection\_covs[d\_idx]

return self.R\_matrix

def \_depth\_robust\_R(track, z, R\_base):

"""Роздуваємо  $\sigma_{Z^2}$  пропорційно квадрату інновації глибини, щоб великий стрибок глибини майже не зсував фільтр (трек тримає глибину прогнозом, латераль X,Y оновлює з валідної детекції).

z\_pred беремо з x\_prior (predict уже викликано для всіх треків)."""

z\_pred = np.dot(track.h\_matrix, track.imm.x\_prior)

depth\_innov = float(z[2] - z\_pred[2])

# Мертва зона: душимо лише надлишок |innov| понад поріг

# нормальної фізики, щоб не калічити легітимну глибину.

excess = max(0.0, abs(depth\_innov) - self.depth\_innov\_free)

R\_eff = np.array(R\_base, dtype=float, copy=True)

```

R_eff[2, 2] += (self.depth_hold_gain * excess) ** 2
return R_eff

for track in self.tracks:
    track.predict()

if len(detections_3d) == 0:
    for track in self.tracks:
        track.mark_missed()
    self._age_spawn_buffer()
    self._manage_lifecycle()
    return

if len(self.tracks) == 0:
    # Особливий випадок: треків ще немає. Будь-яка детекція має
    # шанс на bootstrap, якщо у spawn-буфері є попередник.
    for z in detections_3d:
        self._spawn_track_from_unmatched(z)
    self._age_spawn_buffer()
    return

cost_matrix = np.full((len(self.tracks), len(detections_3d)), 1e5)

for t, track in enumerate(self.tracks):
    # Gate A: прогнозована позиція треку (h·x_prior) — точка відліку
    # для коваріаційно-незалежного residual-стелажу.
    z_pred = np.dot(track.h_matrix, track.imm.x_prior)
    for d, z in enumerate(detections_3d):

```

```

R_d = _R_for(d)
# Gate A: відкидаємо пару, якщо евклідів стрибок від прогнозу
# фізично неможливий за один крок асоціації — НЕЗАЛЕЖНО від
# того, наскільки роздулась P під час коастингу (саме це
# відкривало Mahalanobis-гейт для телепортів fr763/807/86).
if self.enable_physical_gate:
    if self.enable_depth_robust_gate:
        # Gate A-depth: гейтимо ЛИШЕ латераль (X,Y). Велику
        # інновацію глибини не відкидаємо — нею керує
        # depth-distrust (роздутий  $\sigma_Z$  і в гейтингу, і на
        # update). Так детекція з валідним u,v приймається.
        lateral = float(np.hypot(z[0] - z_pred[0],
                                z[1] - z_pred[1]))
        if lateral > self.max_assoc_residual:
            continue
        R_d = _depth_robust_R(track, z, R_d)
    elif np.linalg.norm(z - z_pred) > self.max_assoc_residual:
        continue
    dist_sq = track.get_mahalanobis_distance(z, R_d)
    if dist_sq < self.gating_threshold:
        cost_matrix[t, d] = dist_sq

# 3. Асоціація (Угорський алгоритм)
row_ind, col_ind = linear_sum_assignment(cost_matrix)

unmatched_tracks = set(range(len(self.tracks)))
unmatched_detections = set(range(len(detections_3d)))

```

# 4. Оновлення знайдених пар

for r, c in zip(row\_ind, col\_ind):

if cost\_matrix[r, c] < self.gating\_threshold:

# Зберігаємо Mahalanobis<sup>2</sup> НА МОМЕНТ accept'у — він піде

# у get\_dynamic\_transition\_matrix наступного predict().

self.tracks[r].\_last\_mahalanobis\_sq = float(

cost\_matrix[r, c]

)

R\_c = \_R\_for(c)

if self.enable\_depth\_robust\_gate:

# Та сама роздута  $\sigma_Z$ , що й у гейтингу: депт тримається

# прогнозом, латераль X,Y оновлюється з детекції.

R\_c = \_depth\_robust\_R(self.tracks[r],

detections\_3d[c], R\_c)

self.tracks[r].update(detections\_3d[c], R=R\_c)

unmatched\_tracks.remove(r)

unmatched\_detections.remove(c)

# 5. Обробка пропущених треків

for t in unmatched\_tracks:

self.tracks[t].mark\_missed()

# 6. Створення нових треків для нерозпізнаних детекцій

# (із bootstrap'ом швидкості, якщо знайдено попередника).

# Gate D: якщо реальний Confirmed-трек ЩОЙНО коастиє коротко

# ( $0 < tsu \leq \text{spawn\_suppress\_max\_coast}$ ), не народжуємо конкурента з

# викидних детекцій — нехай реальний трек ре-захопить м'яч. Викид НЕ

# потрапляє і в spawn-буфер → пізніший легітимний спавн стартує з

```

# нульовою швидкістю (без паразитного vz-bootstrap).
suppress_spawn = (
    self.spawn_suppress_max_coast > 0
    and any(
        t.state == 'Confirmed'
        and 0 < t.time_since_update <= self.spawn_suppress_max_coast
        for t in self.tracks
    )
)
if not suppress_spawn:
    for d in unmatched_detections:
        self._spawn_track_from_unmatched(detections_3d[d])

# 7. Старіння spawn-буфера + видалення старих треків
self._age_spawn_buffer()
self._manage_lifecycle()

```

```

# -----
# Допоміжні методи bootstrap'у початкової швидкості
# -----
def _spawn_track_from_unmatched(self, z_now):

```

```

    """

```

Створює новий Track для незасоційованої детекції  $z\_now$ . Якщо у spawn-буфері є точка-попередник, що задовольняє обмеженням (близько у просторі, недавно у часі, узгоджена швидкість) — ініціалізує трек з  $v\_initial = (z\_now - z\_prev) / (\Delta fr \cdot dt)$  та  $hits=2$ ; інакше — звичайна ініціалізація з нулевою швидкістю. Spawn-точка, яку використано для bootstrap'у, видаляється з

буфера (один-до-одного), щоб уникнути «склейки» двох треків з однієї історичної детекції.

»»»»»

```
z_now_arr = np.asarray(z_now, dtype=float)
```

```
best_idx = -1
```

```
best_dist = self.bootstrap_max_dist
```

```
best_v = None
```

```
best_gap = 0
```

```
for i, (z_prev, f_prev) in enumerate(self._spawn_buffer):
```

```
    gap = self._frame_counter - f_prev
```

```
    if gap <= 0 or gap > self.bootstrap_max_gap_frames:
```

```
        continue
```

```
    d_xyz = float(np.linalg.norm(z_now_arr - z_prev))
```

```
    if d_xyz >= best_dist:
```

```
        continue
```

```
    gap_dt = gap * self.dt
```

```
    if gap_dt <= 0:
```

```
        continue
```

```
    v_cand = (z_now_arr - z_prev) / gap_dt
```

```
    speed = float(np.linalg.norm(v_cand))
```

```
    if speed > self.bootstrap_max_speed:
```

```
        # Швидкість за межами фізично можливої — не bootstrap'имо.
```

```
        continue
```

```
    best_idx = i
```

```
    best_dist = d_xyz
```

```
    best_v = v_cand
```

```
    best_gap = gap
```

```

if best_idx >= 0:
    # Bootstrap! Створюємо трек з ненульовою v та hits=2.
    self.tracks.append(
        Track(self.next_id, z_now_arr, self.dt,
            v_initial=best_v, initial_hits=2,
            min_hits=self.min_hits,
            enable_hysteresis=self.enable_hysteresis,
            enable_cov_reset=self.enable_cov_reset,
            enable_coast_cov_cap=self.enable_coast_cov_cap,
            enable_coast_z_fence=self.enable_coast_z_fence,
            coast_z_min=self.coast_z_min,
            coast_z_max=self.coast_z_max,
            hit_residual_min_sq=self.hit_residual_min_sq,
            hit_residual_max_sq=self.hit_residual_max_sq,
            bounce_height_max=self.bounce_height_max,
            bounce_max_coast=self.bounce_max_coast,
            m_hit_target=self.m_hit_target)
    )
    self.next_id += 1
    # Видаляємо використану spawn-точку.
    del self._spawn_buffer[best_idx]
else:
    # Класичний spawn з нульовою швидкістю + кладемо детекцію
    # у буфер для майбутнього bootstrap'у.
    self.tracks.append(
        Track(self.next_id, z_now_arr, self.dt,
            min_hits=self.min_hits,

```



```

        enable_hysteresis=self.enable_hysteresis,
        enable_cov_reset=self.enable_cov_reset,
        enable_coast_cov_cap=self.enable_coast_cov_cap,
        enable_coast_z_fence=self.enable_coast_z_fence,
        coast_z_min=self.coast_z_min,
        coast_z_max=self.coast_z_max,
        hit_residual_min_sq=self.hit_residual_min_sq,
        hit_residual_max_sq=self.hit_residual_max_sq,
        bounce_height_max=self.bounce_height_max,
        bounce_max_coast=self.bounce_max_coast,
        m_hit_target=self.m_hit_target)
    )
    self.next_id += 1
    self._spawn_buffer.append(
        (z_now_arr.copy(), self._frame_counter)
    )

def _age_spawn_buffer(self):
    """Видаляє з буфера записи, старші за bootstrap_max_gap_frames."""
    cutoff = self._frame_counter - self.bootstrap_max_gap_frames
    self._spawn_buffer = [
        (z, f) for (z, f) in self._spawn_buffer if f > cutoff
    ]

def _manage_lifecycle(self):
    active_tracks = []
    for track in self.tracks:
        # Видаляємо трек, якщо він довго не оновлювався

```

```

if track.time_since_update > self.max_age:
    track.state = 'Deleted'

# Дефект 1: привид-трек. Tentative (так і не підтвердився), а вже
# коастиє понад tentative_max_age кадрів → шумовий спавн, який без
# цього малював би 80-кадрову фікцію. Підтверджені треки не зачеплені.
elif (track.state == 'Tentative'
      and track.time_since_update > self.tentative_max_age):
    track.state = 'Deleted'

# Дефект 2: підпідлоговий коаст. Лише коли трек БЕЗ виміру (коаст):
# за наявності детекції у вимірюється й клампінг не потрібен.
elif (track.time_since_update > 0
      and track.imm.x[3] < self.floor_kill_y):
    track.state = 'Deleted'

# Трек залишається, якщо він не видалений
if track.state != 'Deleted':
    active_tracks.append(track)

self.tracks = active_tracks

if self.enable_single_ball_nms:
    self._suppress_duplicate_confirmed()

def _suppress_duplicate_confirmed(self):
    """Phase C: single-ball NMS. Домен — один м'яч у грі, тож
    одночасно має бути лише ОДИН Confirmed-трек. Якщо їх кілька

```

(фантом-дублі від сміттєвих спавнів або дивергентного коастингу),  
лишаємо найкращий, решту ВИДАЛЯЄМО.

Видалення (а не пониження до Tentative) — навмисне: понижений дубль продовжує красти детекції в Угорському алгоритмі (на маневрі детекція перескакує між двома треками, що тримають той самий м'яч), re-confirm'ється і знову конкурує — дубль жив ~200 кадрів і давав 9-14 стрибків ідентичності. Видалення зупиняє це: детекція стабільно йде на переможця; якщо ж переможець справді розходиться з м'ячем, нова детекція спавнить свіжий трек (ціна — трохи вища fragmentation, але траєкторія помітно гладша: jerk↓, switches 9→3).

Найкращий = (min bucketed-tsu, max hits, min останній Mahalanobis^2): активно оновлюваний > довгоживучий > добре вписаний.

ВАЖЛИВО — bucketing tsu: короткий коастинг ( $tsu \leq TSU\_TOL$ ) прирівнюється до 0. Без цього один пропущений кадр на реальному треку ( $tsu=1$ ) віддавав би ідентичність дивергентному привиду з  $tsu=0$  — навіть якщо привид має 4 hits проти 245 у реального.

Це спричиняло мерехтіння ідентичності під час маневру (детекції на 2-3 кадри перескакували на привид-трек). З bucket'ом реальний трек (на порядок більше hits) тримає ідентичність крізь коротку турбулентність; привид із ДОВГИМ коастингом ( $tsu > TSU\_TOL$ ) усе ж поступається активному треку — коректний handoff.””””

```
confirmed = [t for t in self.tracks if t.state == 'Confirmed']
```

```
if len(confirmed) < 2:
```

```
    return
```

```

def _key(t):
    eff_tsu = 0 if t.time_since_update <= self.SINGLE_BALL_NMS_TSU_TOL
    else t.time_since_update
    return (eff_tsu, -t.hits, t._last_mahalanobis_sq)

confirmed.sort(key=_key)
winner = confirmed[0]

# Merge-identity (фікс fragmentation): домен — один м'яч, тож усі
# співіснуючі Confirmed-треки = той САМИЙ фізичний м'яч, лише
# роздроблений маневром (детекція вистрибує з гейту → народжується
# конкурент). Переможець успадковує МІНІМАЛЬНИЙ id групи →
# логічна ідентичність неперервна через шов, а не новий id щоразу.
# Розділені реальним розривом траєкторії (різні розіграші) НІКОЛИ
# не співіснують як Confirmed (gap десятки кадрів) → сюди не
# потрапляють → хибного злиття немає. Переносимо й МАКС hits, щоб
# злитий трек лишався "досвідченим" і стабільним переможцем у
# наступних NMS (tiebreaker -hits).
#
# Gate B (merge guard): id успадковуємо ЛИШЕ від фізично примирених
# членів групи ( $\leq$  SINGLE_BALL_MERGE_MAX_DIST від переможця). Далек
# FP-трек, що дожив до Confirmed (пройшов min_hits-верифікацію), усе
# одно НЕ донорить свій id — інакше merge-identity легалізувала б
# телепорт (ампліфікація проблеми fr763). Він просто видаляється
# як дубль. Маневр-фрагменти лишаються близько ( $\leq \sim 2$  м) → зливаються
# нормально; реальний м'яч, що далеко "перестрибнув", дає новий id —

```

# коректно, бо такий стрибок = фактично новий сегмент гри.

```
def _pos(t):
```

```
    x = t.imm.x_post
```

```
    return np.array([x[0], x[3], x[6]])
```

```
wp = _pos(winner)
```

```
reconcilable = [
```

```
    t for t in confirmed
```

```
    if np.linalg.norm(_pos(t) - wp) <= self.SINGLE_BALL_MERGE_MAX_DIST
```

```
]
```

```
winner.track_id = min(t.track_id for t in reconcilable)
```

```
winner.hits = max(t.hits for t in reconcilable)
```

```
for loser in confirmed[1:]:
```

```
    loser.state = 'Deleted'
```

```
self.tracks = [t for t in self.tracks if t.state != 'Deleted']
```

```
def get_confirmed_tracks(self):
```

```
    """Повертає позиції лише тих треків, в яких ми впевнені"""
```

```
    return [t for t in self.tracks if t.state == 'Confirmed']
```

### **training\_models/model\_train\_remote.py**

```
import os
```

```
import torch as t
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
from pathlib import Path
```

```
from torchvision import transforms
```

```

from torchvision.transforms import Compose, ToTensor, PILToTensor, Resize
from tqdm import tqdm
import numpy as np
# import optuna as opt
from ultralytics import YOLO
import albumentations as A

```

```

main_model = YOLO('../parsing_datasets_scripts/yolo26m.pt')

```

```

custom_transforms = [
    A.MotionBlur(blur_limit=(7, 15), p=0.4),
    A.GaussNoise(std_range=(0.01, 0.05), p=0.2),
    A.CLAHE(clip_limit=4.0, p=0.3),
    A.RandomBrightnessContrast(brightness_limit=0.25,
                               contrast_limit=0.25, p=0.4),
    A.HueSaturationValue(hue_shift_limit=15, sat_shift_limit=30,
                         val_shift_limit=20, p=0.3),
]

```

```

# Purely for server training

```

```

main_model.train(data='/workspace/diploma_project/data/server_data.yaml',
                 epochs=100, device='0', imgsz=640,
                 augmentations=custom_transforms, patience=20)

```

```

metrics = main_model.val(data='/workspace/diploma_project/data'
                        '/server_data.yaml', imgsz=640)

```

```
print(f'mAP50: {metrics.box.map50}')
print(f'mAP50-95: {metrics.box.map95}')
```